Source: docs\FIREBASE_SETUP.md | Export: 11 November 2026

Firebase connection: Step 1

> Subsequent administrator-access work is documented in ADMIN_ACCESS.md. Real Firebase Auth and a server-managed permission listener now apply to `/admin`. The owner has supplied the previously pending deployed-rules snapshot; its permissive test-mode grant must be replaced through the owner's reviewed rule deployment. The Step 1 document below records the original connection milestone and developer probe behavior; its earlier test counts and Auth scope are historical.

Target and verified status

- Existing Firebase project: `parinaya-silks`.
- Database: **Cloud Firestore**, ID **`(default)`**, confirmed by the owner.
- Web application: the owner's supplied Firebase web app configuration.
- Database connection: Firebase modular browser SDK, using Firestore Lite for online server reads. `initializeFirestore(app, { host: 'firestore.googleapis.com', ssl: true })` selects the confirmed default instance and explicitly targets hosted Firestore; no database is created. A cache retained through Vite HMR reuses the client.
- Current status: local configuration validation, app/client initialization, HMR reuse and controlled read-result handling verified. **Live database read permissions remain unverified.** The owner has no known permitted signed-out document path and will supply current rules. No live document was queried during this step so far.
- No local rules files were present. No deployed rules were read through privileged access or changed. Do not change rules simply to make a diagnostic succeed.

The production preparation gate and local demo workflows remain intact. The catalog, orders, customer profile, authentication and checkout continue to use the existing isolated demo service. Firebase initialization does not migrate data or confer admin access. Authentication, CRUD, payments and permission implementation are outside Step 1.

Files and responsibilities

File	Purpose
`src/services/firebase/config.ts`	Validates the six browser app variables; rejects missing/placeholder/decorated values, wrong project and inconsistent sender/app IDs. Errors contain variable names and reasons only.
`src/services/firebase/app.ts`	Lazily initializes a named Firebase app once, reuses it across HMR, and detects changed configuration without logging values.
`src/services/firebase/firestore.ts`	Validates the default database target and reuses the Firestore Lite client. No emulator connection, reads, queries or writes on initialization.
`src/services/firebase/verification.ts`	Explicit one-document server-read check; returns existence/status only. No collection scans, document data, fixture fallback or writes.
`src/services/firebase/developer.ts`	Installs `window.parinayaFirebase` only in Vite development; omitted from production. No customer-facing panels or public debug endpoint.
`src/main.tsx`	Initializes app/client without reading Firestore or changing UI. Loads developer controls in development only.
`scripts/firebase-check.ts`	Developer CLI using Vite environment loading and the same SDK/client/check implementation.
`scripts/firebase-verify-build.ts`	Compares compiled public config against expected environment values locally or at an explicitly supplied deployment URL. Prints names/booleans only. Never reads Firestore or evaluates downloaded JavaScript.
Firebase test files / `scripts/firebase-browser.test.py`	Config, SDK reuse, paths, deadlines, error classification and intercepted browser SDK read tests.
`.env.example` / ignored `.env.local`	Placeholder template / supplied local web app configuration.

File	Purpose
`package.json` / `package-lock.json`	Firebase dependency, compatible app-package pins and developer commands.

Firebase is pinned to `12.19.0`. Its `@firebase/app` and app-compat packages are fixed to the versions expected by that release (`0.16.2` and `0.5.18`): their broad peer ranges otherwise selected newer Node-24-only packages during installation. The installed app/Firestore packages support the existing Node 22 runtime. Only app and Firestore Lite services are imported; installing the SDK does not activate Analytics, Auth or other products.

Exact environment variables

Use these names in `.env.local`, Vercel **Production**, and Vercel **Preview** if previews should use this same explicitly approved project:

Variable	Value
`VITE_FIREBASE_API_KEY`	Supplied Firebase browser API key; already saved locally, not repeated in documentation.
`VITE_FIREBASE_AUTH_DOMAIN`	`parinaya-silks.firebaseapp.com`
`VITE_FIREBASE_PROJECT_ID`	`parinaya-silks`
`VITE_FIREBASE_STORAGE_BUCKET`	`parinaya-silks.firebasestorage.app`
`VITE_FIREBASE_MESSAGING_SENDER_ID`	Supplied numeric sender ID; copy from local configuration.
`VITE_FIREBASE_APP_ID`	Supplied web app ID; copy from local configuration.
`VITE_FIREBASE_DATABASE_ID`	`(default)`
`VITE_FIREBASE_CHECK_DOCUMENT_PATH`	**Optional; leave empty for now.** Set only after rules review identifies one approved nonsensitive signed-out document path. Never set to a collection or root.

These are browser-visible web configuration, not server secrets. Vite embeds `VITE_` values in client JavaScript. Do not put service-account JSON, private keys, Admin SDK credentials, payment secrets or other server credentials in these variables. Firebase browser API keys identify the project; they do not authorize database access. See Firebase's API-key guidance.

No emulator environment variables are implemented or read. Production always targets Firebase's hosted default Firestore instance. The supplied storage bucket is validated as app metadata; no Storage client is connected.

The tracked-file scan found no private-key/service-account patterns or literal browser API keys in tracked sources. Only `.env.example` is tracked; `.env.local` is confirmed ignored. This is a scoped pattern scan, not a full historical secret audit.

Local development

The supplied values are already in ignored `.env.local`. For a new checkout, copy `.env.example` to `.env.local` and fill the three supplied placeholders (API key, sender ID, app ID). Do not overwrite an existing local environment file blindly.

```
npm.cmd ci
npm.cmd run dev
```

Restart Vite and reload after changing environment values. App configuration changes during HMR require a full reload so the existing app cannot silently retain a previous target.

At `http://127.0.0.1:3000`, open browser developer tools → Console:

```
window.parinayaFirebase.status()
await window.parinayaFirebase.check()
```

Expected now: status reports `database-initialized`, target `parinaya-silks` / `(default)`, `appInitialized: true`, and `databaseReadVerified: false`. The second call returns `check-path-missing`; it sends no read request. Initialization is not evidence of authorized database access.

After existing rules have established a safe path, call `check('approved_collection/approved_document')` or set that exact path in the optional environment variable and restart. The identifier above is a placeholder, not a real/approved document. The validator accepts collection/document pairs with simple letters, digits, hyphens and underscores, up to five pairs. It rejects roots, collection-only paths, URLs, wildcards and query syntax.

The equivalent developer CLI is:

```
npm.cmd run firebase:check
# Only after a real safe path is identified:
npm.cmd run firebase:check -- --document approved_collection/approved_document
```

Without a path the CLI exits nonzero. It also exits nonzero for configuration problems, denied access, network errors and timeouts. An existing document or a server-confirmed absent document produces a verified read result. The tool returns no document fields.

Interpreting outcomes

Status	Meaning
`missing`, `invalid`	Required app config absent or rejected; no app initialized by that attempt.
`initialization-error`	SDK app failed to initialize or config changed relative to the existing app.
`database-config-missing`, `database-config-invalid`	Required database ID absent or different from the confirmed default.
`database-initialized`	App and client exist; database access is still unverified.
`check-path-missing`, `check-path-invalid`	No safe document path was provided; no read was attempted.
`read-succeeded`	Server read succeeded and the exact document exists.
`read-empty`	Server read succeeded and confirmed that exact document does not exist. No sample data is substituted.
`permission-denied`	The read was rejected; this is not an authorized-read success. Review rules and relevant Firebase API/App Check restrictions without weakening them.
`authentication-required`	An unauthenticated request was rejected; authentication remains outside Step 1.
`network-error`, `service-error`, `read-error`	Transport/service/other error; no authorized-read success. Document-not-found returned as an SDK service error is not assumed to mean the whole database is empty.
`timeout`	No read success was established within ten seconds.
`check-in-progress`	A prior SDK read has not settled; no extra read was started.

The application invokes only one `getDoc` for one explicit path and adds no retry loop. Firestore Lite always uses the server, so cache cannot generate a false successful connection result. The SDK's underlying request cannot be cancelled by this wrapper: a timeout bounds the diagnostic wait, and further checks are blocked until that operation settles. A late response never changes the returned timeout into success.

Existing rules may intentionally deny all reads while auth is absent. A denial is an accurate integration outcome, not a reason to open access. Do not invent a public health document, create a test record, seed products, grant public writes or deploy rules for this step.

Vercel Production and Preview setup

The owner performs these steps; no Vercel configuration was changed by this task:

1. Open the existing Vercel project → Settings → Environment Variables.
2. Add all seven required variables from the table, copying the exact supplied browser configuration from `.env.local`. Select **Production**. Keep the optional check path empty until approved.
3. If preview deployments are used, configure the same variables under **Preview** deliberately. There is no automatic alternate-project fallback. If previews should use another database later, that requires a separate explicit target decision and guard update.
4. Run the normal production build command `npm run build`, with Vite output directory `dist`. Do not switch the production deployment to `build:demo` or `npm run dev`.
5. Trigger a fresh build/deployment after changing variables. Vite client variables are resolved at build time; changing the Vercel settings does not update an already-built deployment. See Vercel environment variables.

If using the Vercel CLI with the correct already-linked project, the owner can pull the configured Production values into an ignored file:

```
vercel env pull .env.production.local --environment=production
npm.cmd run firebase:check -- --mode production
npm.cmd run build
npm.cmd run firebase:verify-build
```

This downloads the project's environment; keep that local file private, inspect neither values in logs nor tracked diffs, and do not commit it. Shell environment variables take precedence over dotenv values; `.env.production.local` takes precedence over generic local files. These commands verify **Production configuration locally**, not the hosted deployment's access. See Vercel CLI env.

Verifying a deployed build without exposing app diagnostics

After the owner deploys and provides the exact final deployment URL, run locally with the expected Production configuration loaded:

```
npm.cmd run firebase:verify-build -- --url https://YOUR-EXACT-DEPLOYMENT-URL
```

This reads the public index and its entry JavaScript, compares each expected browser value without printing it, and reports names/booleans only. Use a direct final URL: redirects, protected previews, missing scripts and cross-origin entrypoints are rejected. For this repository's current Vite bundle layout, it verifies the served compiled config. If future chunk splitting moves config out of the entry script, an unmatched check requires inspection; it must not be treated as a successful match. **Static config matching is not a database read.**

On the deployed page, confirm the preparation page remains and `typeof window.parinayaFirebase` is `'undefined'`. There is intentionally no public diagnostic route, endpoint or status panel. No Firestore request occurs on page load, so an empty Network tab does not prove connectivity.

Once a safe document path is confirmed by rules review, an owner can perform one optional read from the deployed origin using developer tooling. The following Console snippet uses Firebase's hosted browser SDK in that browser session only; it does not install a debug endpoint or modify the deployed app. Replace both placeholders. Use the same browser API key verified by the build comparison. No customer record path should be used:

```
const appSdk = await import('https://www.gstatic.com/firebasejs/12.19.0/firebase-app.js');
const dbSdk = await import('https://www.gstatic.com/firebasejs/12.19.0/firebase-firestore-lite.js');
const temporaryApp = appSdk.initializeApp({
  apiKey: 'PASTE_SUPPLIED_BROWSER_API_KEY',
  projectId: 'parinaya-silks'
}, 'parinaya-owner-read-check-' + Date.now());
const temporaryDb = dbSdk.initializeFirestore(temporaryApp, { host: 'firestore.googleapis.com', ssl: true });
let checkTimer;
const read = dbSdk.getDoc(dbSdk.doc(temporaryDb, 'APPROVED_COLLECTION/APPROVED_DOCUMENT'))
  .then(snapshot => ({ status: snapshot.exists() ? 'read-succeeded' : 'read-empty', databaseReadVerified: true }))
  .catch(error => ({ status: error.code === 'permission-denied' ? 'permission-denied' : 'read-error', databaseReadVerified: false }));
const result = await Promise.race([
  read,
  new Promise(resolve => { checkTimer = setTimeout(() => resolve({ status: 'timeout', databaseReadVerified: false }), 10000); })
]);
clearTimeout(checkTimer);
console.log(result); // Existence/status only, never snapshot.data().
await dbSdk.terminate(temporaryDb);
await appSdk.deleteApp(temporaryApp);
```

This verifies the chosen database access from that deployed origin with a separately initialized browser client. Together with static config matching it checks the target and origin access; it does not exercise product/order features, which remain disconnected. A denied, blocked CDN import, network error or timeout is not a successful database read. Do not weaken CSP, API restrictions or rules to obtain success. Hosted verification has not been performed by this task.

Validation performed and remaining blocker

- TypeScript passes.
- 24 unit tests pass, including config failures, actual SDK app/client reuse, bounded waits, denied/network/service results and invalid paths.
- Default Vite production build passes and compiled config matches the supplied local values; comparison emits no values.
- Python Playwright with intercepted Firestore transport passes existing/missing/denied single-document reads through the real browser SDK. No fixture request reaches Firebase. These tests verify code/SDK behavior, **not deployed rule correctness**.
- Browser HMR-style reimports reuse one app. Fixture reads still target hosted Firestore when ambient SDK emulator defaults are deliberately injected. Page startup sends zero Firestore reads. Production preview preserves the preparation gate and exposes no developer global. Production assets omit the diagnostic read verifier.
- No actual live read, emulator rule test, deployed-rule inspection, Vercel environment update, deployment, data write or migration occurred. The current next dependency is the owner's existing Firestore rules and a safe read path, or confirmation that all unauthenticated reads are intentionally denied. Until then, the Step 1 implementation is prepared but live authorized access remains unverified.